Dataset: TraceLab/README.md at main · uw-syfi/TraceLab

Week 3 Progress Update — TraceLab Workload Analysis

What I worked on this week

I finished the main Week 3 workload analysis using the TraceLab coding-agent dataset.

The goal for Week 3 was basically:

> **Understand what the tool-use workload actually looks like and determine whether there is enough variability in tool execution times to motivate prediction-aware KV-cache retention.**

I downloaded the TraceLab DuckDB dataset and analyzed the tool-call data.

Dataset

The current TraceLab dataset contains:

- **8,058 sessions**
- **52 users**
- **665,453 LLM rounds**
- **743,819 tool calls**
- **600,272 tool calls with recorded latency**

The TraceLab overview reports 600,272 calls with latency. For the specific Week 3 wall-latency analysis, I filtered for non-null, non-negative `tool_wall_latency_ms` values.

The dataset contains tables such as:

- `rounds`
- `tool_calls`
- `timing_events`

For our initial workload analysis, I used `tool_wall_latency_ms`, which represents:

> time from when a tool call is emitted until its result is returned.

So our current tool-level duration is:

$$D = t_{\text{result}} - t_{\text{emitted}}$$

How I analyzed the data

> I used the `tool_calls` table as the main source for the Week 3 tool-latency analysis. I focused on `tool_wall_latency_ms`, which measures the elapsed time between a tool call being emitted and its result being returned.

For the latency analysis, I used tool calls with a non-null, non-negative `tool_wall_latency_ms`. I did not remove legitimate long-running calls simply because they had large durations, since the long tail is relevant to our research question.

I initially examined the workload across all tool types, then focused the main latency analysis on automated execution tools:

- `Bash`
- `exec_command`
- `write_stdin`
- `exec`
- `shell_command`

Main finding: tool latency has a long tail

The important result is that **most tool calls can be very short, but some calls take a very long time.**

For the main automated tools:

Tool	Median	P90	P95	P99
Bash	0.329 s	20.404 s	70.906 s	300.136 s
exec_command	0.265 s	1.600 s	5.238 s	30.172 s
write_stdin	5.006 s	44.715 s	90.008 s	300.002 s
exec	1.457 s	10.050 s	10.058 s	10.078 s
shell_command	0.051 s	0.867 s	1.767 s	19.167 s

This shows that tool duration varies **a lot**, especially for Bash and write_stdin.

For example, Bash has a median duration of only **0.329 seconds**, but its:

- P90 = **20.4 seconds**
- P95 = **70.9 seconds**
- P99 = **300.1 seconds**

So a tool call that is usually very short can sometimes run for minutes.

Long calls are not extremely rare

Some additional numbers:

Bash

- **20.10%** take >5 seconds
- **14.04%** take >10 seconds
- **8.36%** take >30 seconds
- **5.72%** take >60 seconds

write_stdin

- **58.99%** take >5 seconds
- **34.99%** take >10 seconds
- **25.49%** take >30 seconds
- **9.07%** take >60 seconds

This is important because long-duration calls are not limited to a handful of extreme observations.

Note that these percentages should be interpreted as **tool-call-level statistics**, not as percentages of entire agent sessions.

What this means for our project

The Week 3 results support the motivation for our project:

> **Tool durations are highly variable. The observed variability motivates investigating whether a fixed TTL is sufficient, or whether information about expected tool duration can improve KV-cache retention decisions.**

For example, if we use a very short TTL, we may evict a cache while a long-running tool is still executing. If we use a very long TTL, we may unnecessarily keep caches for many tools that finish quickly.

However, **we have NOT yet shown that our prediction-aware policy is better.**

Week 3 only establishes that the workload has the variability that makes prediction potentially useful.

Candidate Prediction features

I investigated what information could potentially be available **when a tool call is issued**, because we must avoid using information that becomes available only after the tool finishes.

Potential features include:

- tool name
- provider
- model
- input size (`input_chars`)
- command type / `command_skeleton`
- executable types (`executables`)
- round/session information
- previous tool durations

- previous tool history

These are **candidate features**, not a finalized feature set. Week 4 will determine which features are useful and whether any introduce temporal or session-level information leakage.

We specifically checked TraceLab's source code and confirmed that `command_skeleton` and `executables` are derived from the **original tool input/command**, rather than from the tool's result.

Therefore, under our modeling assumption that the tool command is known when the agent issues the tool call, these are reasonable candidates for prediction features.

For predicting the duration of the **same tool call**, we cannot use variables that are only known after the tool executes. Examples include:

- `result_chars`
- `result_at`
- `tool_wall_latency_ms`
- `tool_internal_latency_ms`
- `command_status`
- `command_exit_code`

Those are either only known after the tool runs or directly describe the actual duration/result.

Important caveat

There is a distinction between **individual tool-call duration** and a complete command lifecycle.

For example, Codex can have:

exec_command
↓
write_stdin
↓
write_stdin
↓
finished

So we shouldn't automatically assume that every individual tool-call latency represents the entire period for which an agent's KV cache would need to be retained.

For Week 3, I used individual `tool_wall_latency_ms` as the common tool-level measure because it is available consistently across the dataset. This is sufficient for initial workload characterization, but it does **not necessarily represent the full period for which an agent's KV cache would need to be retained**. For the later simulator, we may need to reconstruct longer command lifecycles where appropriate, particularly for Codex-style command execution.

What Week 3 established

Established

- Tool execution durations are highly variable and strongly right-skewed.
- Some automated tools have substantial long-duration tails.
- Different tool types have very different latency distributions.
- The TraceLab records contain several candidate features that may be available when a tool is issued.
- `command_skeleton` and `executables` are derived from the original tool input and can therefore be considered candidate decision-time features.

Not established yet

- Whether tool duration can be predicted accurately.
- Whether prediction is better than simple baselines.
- Whether prediction-aware retention improves KV-cache performance.
- How much prediction error the retention policy can tolerate.
- Whether the same conclusions hold under different GPU-memory constraints.

Week 3 conclusion

The TraceLab workload contains substantial and highly variable tool execution times, with particularly heavy tails for tools such as Bash and `write_stdin`. This provides a realistic setting for investigating whether future tool duration can be predicted and whether those predictions can improve KV-cache retention decisions.

Week 3 therefore establishes the workload motivation, but not yet the effectiveness of the proposed method.

Next step: Week 4 — Duration Prediction

> Week 4 will investigate whether tool duration can be predicted using information available when the tool call is issued.

Initial prediction approaches

1. **Global median:** predict every tool duration using the median duration across the training data.
2. **Tool-specific median:** predict duration using the median for the corresponding tool type.
3. **Feature-based model:** use decision-time features such as tool name, provider, model, input size, command type, and available history.

Evaluation

We will measure how accurately these approaches predict the actual tool duration.

Because the workload is highly right-skewed, we should not rely on a single metric such as RMSE. We will consider metrics appropriate for the duration distribution, such as absolute error and log-scale error, and examine prediction error across different parts of the distribution.

Main Week 4 question

How predictable is tool duration using information available at the time the tool call is issued?

Important

Prediction accuracy is not the final objective. The purpose of the predictor is to provide information to the KV-cache retention policy. Later weeks will test whether even imperfect predictions actually improve retention decisions.